

Query 23A — reconstruir transiciones

DROP TABLE IF EXISTS

bu_frd.model_trades_transition_13m_2022_2025;

CREATE TABLE

bu_frd.model_trades_transition_13m_2022_2025

USING PARQUET

AS

WITH normalized AS (

SELECT

```
/* =====  
COHORT / TIME  
===== */
```

```
party_id,  
observation_date,  
observation_year,  
UPPER(model_split) AS model_split,  
label_3m,  
cohort_role,
```

```
feature_month,  
month_offset,  
feature_window,
```

```
/* =====  
ACCOUNT  
===== */
```

```
acct_id,
```

```
data_date,  
eff_dte,
```

```
/* Keep original value */  
acct_mop_cde,
```

```
/* Normalized value used for comparisons */  
UPPER(  
  TRIM(  
    CAST(acct_mop_cde AS STRING)  
  )  
) AS current_mop,
```

acct_typ_cde,
porf_typ_cde,
kob_posn_1_cde,
kob_posn_2_cde,

open_dte,
clsd_dte,
paid_out_dte,

curr_bal_amt,
cr_lmt_amt,
high_cr_amt,
past_due_amt,

payt_amt,
payt_dte,

/* =====
CURRENT DELINQUENCY LEVEL
===== */

CASE
WHEN UPPER(TRIM(CAST(acct_mop_cde AS STRING))) = '01'
THEN 0
WHEN UPPER(TRIM(CAST(acct_mop_cde AS STRING))) = '02'
THEN 1
WHEN UPPER(TRIM(CAST(acct_mop_cde AS STRING))) = '03'
THEN 2
WHEN UPPER(TRIM(CAST(acct_mop_cde AS STRING))) = '04'
THEN 3
WHEN UPPER(TRIM(CAST(acct_mop_cde AS STRING))) = '05'
THEN 4
WHEN UPPER(TRIM(CAST(acct_mop_cde AS STRING))) = '06'
THEN 5
ELSE NULL
END AS current_delinquency_level

FROM
bu_frd.model_trades_behavioral_final_13m_2022_2025
,

lagged AS (

SELECT

*,

/* =====

PREVIOUS ACCOUNT STATE

===== */

```
LAG(current_mop) OVER (  
  PARTITION BY  
    party_id,  
    observation_date,  
    acct_id  
  ORDER BY data_date  
) AS previous_mop,
```

```
LAG(eff_dte) OVER (  
  PARTITION BY  
    party_id,  
    observation_date,  
    acct_id  
  ORDER BY data_date  
) AS previous_eff_dte,
```

```
LAG(curr_bal_amt) OVER (  
  PARTITION BY  
    party_id,  
    observation_date,  
    acct_id  
  ORDER BY data_date  
) AS previous_balance,
```

```
LAG(cr_lmt_amt) OVER (  
  PARTITION BY  
    party_id,  
    observation_date,  
    acct_id  
  ORDER BY data_date  
) AS previous_credit_limit,
```

```
LAG(past_due_amt) OVER (  
  PARTITION BY  
    party_id,  
    observation_date,  
    acct_id  
  ORDER BY data_date  
) AS previous_past_due,
```

```
LAG(current_delinquency_level) OVER (  
  PARTITION BY  
    party_id,  
    observation_date,  
    acct_id
```

```

        ORDER BY data_date
    ) AS previous_delinquency_level

FROM normalized
)

SELECT

    *,

    /* =====
    ACTIVE ACCOUNT

    Based on lifecycle as known at data_date.
    ===== */

    CASE
        WHEN
            (open_dte IS NULL OR open_dte <= data_date)

            AND
            (clsd_dte IS NULL OR clsd_dte > data_date)

            AND
            (paid_out_dte IS NULL OR paid_out_dte > data_date)

        THEN 1
        ELSE 0
    END AS active_account_flag,

    /* =====
    EFFECTIVE UPDATE

    eff_dte changing means the provider's
    verification date changed from the prior
    observed monthly snapshot.
    ===== */

    CASE
        WHEN previous_eff_dte IS NULL
            THEN 0

        WHEN eff_dte <> previous_eff_dte
            THEN 1

        ELSE 0
    END AS effective_update_flag,

```

```
/* =====
DPD DETERIORATION
Only 01-06 participate in ordinal comparison.
===== */
```

```
CASE
  WHEN previous_delinquency_level IS NOT NULL
    AND current_delinquency_level IS NOT NULL
    AND current_delinquency_level >
      previous_delinquency_level

  THEN 1
  ELSE 0
END AS dpd_deterioration_flag,
```

```
/* =====
DPD IMPROVEMENT
===== */
```

```
CASE
  WHEN previous_delinquency_level IS NOT NULL
    AND current_delinquency_level IS NOT NULL
    AND current_delinquency_level <
      previous_delinquency_level

  THEN 1
  ELSE 0
END AS dpd_improvement_flag,
```

```
/* =====
MAGNITUDE OF DPD MOVEMENT

Positive = deterioration
Negative = improvement
===== */
```

```
CASE
  WHEN previous_delinquency_level IS NOT NULL
    AND current_delinquency_level IS NOT NULL

  THEN
    current_delinquency_level
    - previous_delinquency_level
```

```
ELSE NULL
END AS dpd_level_change,
```

```
/* =====
ENTERED ACTIVE ADVERSE STATUS
===== */
```

```
CASE
  WHEN current_mop IN (
    '09','9B','8A','8B'
  )

  AND (
    previous_mop IS NULL
    OR previous_mop NOT IN (
      '09','9B','8A','8B'
    )
  )

  THEN 1
  ELSE 0
END AS entered_active_adverse_flag,
```

```
/* =====
ADVERSE RESOLVED
===== */
```

```
CASE
  WHEN current_mop IN ('9P','8P')

  AND previous_mop IS NOT NULL

  AND previous_mop <> current_mop

  THEN 1
  ELSE 0
END AS adverse_resolved_flag,
```

```
/* =====
BECAME UNRATED
===== */
```

```
CASE
  WHEN current_mop = 'UR'
```

```

        AND previous_mop IS NOT NULL

        AND previous_mop <> 'UR'

    THEN 1
    ELSE 0
END AS became_unrated_flag,

/* =====
LEFT UNRATED
===== */

CASE
    WHEN previous_mop = 'UR'

        AND current_mop <> 'UR'

    THEN 1
    ELSE 0
END AS left_unrated_flag

FROM lagged;

```

QA 23A.1 — la tabla debe conservar el mismo número de filas

```

SELECT
    COUNT(*) AS transition_rows,

    COUNT(DISTINCT party_id)
    AS parties,

    COUNT(DISTINCT acct_id)
    AS accounts,

    MIN(data_date)
    AS min_data_date,

    MAX(data_date)
    AS max_data_date

FROM
    bu_frd.model_trades_transition_13m_2022_2025;

```

QA 23A.2 — revisar los MOP reales

```

SELECT

    current_mop,

    COUNT(*) AS rows,

    ROUND(
        100.0 * COUNT(*)
        / SUM(COUNT(*)) OVER(),
        4
    ) AS pct_rows

FROM
    bu_frd.model_trades_transition_13m_2022_2025

GROUP BY current_mop

ORDER BY rows DESC;

```

QA 23A.3 — conteo de transiciones

```

SELECT

    model_split,
    label_3m,

    COUNT(*) AS account_month_rows,

    SUM(dpd_deterioration_flag)
    AS dpd_deteriorations,

    SUM(dpd_improvement_flag)
    AS dpd_improvements,

    SUM(entered_active_adverse_flag)
    AS entered_active_adverse,

    SUM(adverse_resolved_flag)
    AS adverse_resolved,

    SUM(became_unrated_flag)
    AS became_unrated,

    SUM(left_unrated_flag)
    AS left_unrated

FROM
    bu_frd.model_trades_transition_13m_2022_2025

```

WHERE feature_window <> 'SEED'

GROUP BY
 model_split,
 label_3m

ORDER BY
 model_split,
 label_3m DESC;

QA 23A.4 — revisar magnitud de los DPD jumps

```
SELECT  
  
    dpd_level_change,  
  
    COUNT(*) AS rows  
  
FROM  
    bu_frd.model_trades_transition_13m_2022_2025  
  
WHERE  
    feature_window <> 'SEED'  
  
AND dpd_level_change IS NOT NULL  
  
GROUP BY dpd_level_change  
  
ORDER BY dpd_level_change;
```

QA 23A.5 — revisar `effective_update_flag`

```
SELECT  
  
    feature_window,  
  
    COUNT(*) AS rows,  
  
    SUM(effective_update_flag)  
        AS effective_updates,  
  
    ROUND(  
        100.0 *
```

```
        SUM(effective_update_flag)
        / COUNT(*),
        2
    ) AS effective_update_pct

FROM
    bu_frd.model_trades_transition_13m_2022_2025

GROUP BY feature_window

ORDER BY feature_window;
```